Autonomous Preservation Robots in an Interplanetary Museum Vault

Seyed Morteza Mojtabavi, seyed.mojtabavi@unitn.it, 256328

Date: March 22, 2026

University of Trento
Department of Information Engineering and Computer Science
Via Sommarive 9, 80123
Povo (TN), Italy

1 Scenario Understanding and Justification

This project addresses the Interplanetary Museum Vault (IMV) space scenario. The IMV is a subterranean Martian structure storing fragile artifacts from various space missions. Due to seismic activity and pressure fluctuations, an autonomous robotic curator must relocate high-value artifacts to a safe Stasis-Lab while managing environmental hazards and limited resources.

Scenario Choice Justification: The space scenario was selected because it naturally introduces complex, multi-layered constraints. These include topological hazards (low-pressure zones), strict sequential processing requirements (cooling temperature-sensitive items), and finite physical resources (Anti-Vibration Pods). These constraints provide a robust testbed for comparing how different planning paradigms (Classical, Hierarchical (HTN), and Temporal) handle resource bottlenecks and negative preconditions.

2 Design Choices and Technical Approach

The domain was iteratively expanded from PDDL 2.1 Level 1 (STRIPS/Typing) to HDDL (HTN) and PDDL 2.1 Level 3 (Durative Actions).

2.1 Modeling Strategies

- **Type Hierarchy:** I utilized strict typing (location, agent, spot, artifact, pod) with robot and drone sub-typed under agent. This reduces the number of ground actions and prevents parameter errors during grounding.
- **The Slot System:** Agent capacity is modeled using discrete spot objects linked to agents via the belongs-to predicate. This allows for dynamic capacity (for example, the curator has 2 spots, the drone has 1) without hardcoding separate actions for different loads.
- **Positive Antonyms:** In the temporal domain (Problem 4), I replaced negative preconditions with positive antonyms like normal-pressure and sealing-off. This ensures compatibility with temporal planners like Optic and TFD, which can struggle with intervals containing negative conditions.
- **Concurrency Mutex:** I introduced an available predicate for durative actions. This acts as a mutex lock, requiring an agent to be available at start and restoring it at end, preventing a single agent from performing overlapping tasks while allowing multi-agent parallelization.

2.2 Action and Method Design

- **Universal Quantification (forall):** The unload-pod action uses a forall check to ensure every spot belonging to a robot is an empty-spot before dropping the pod. This prevents the physical paradox of a robot dropping protective gear while still carrying a delicate artifact.
- **HTN Task Decomposition:** In HDDL (Problem 3), methods were structured to enforce specific workflows. For example, the evacuate-hot task forces the agent to visit the cryo-chamber before the stasis-lab. This embeds specific domain knowledge directly into the planner, bypassing the need for it to guess the correct topological order.

3 Deployment and Building

The complete deployment infrastructure, including environment configuration and automated execution scripts for both planutils and ROS 2 PlanSys2, is hosted at the following repository:

`https://github.com/MorteZ76/Automated-Planning-IMV`

The repository provides a modularized Docker approach, separating the static planning analysis from the real-time ROS 2 Humble execution environment.

4 Assumptions and Deviations

- **Artifact Finality:** It is assumed that once an artifact is deposited in the StasisLab, it is permanently secured. The unload-artifact action removes the "carried" status but does not restore the free-artifact status, preventing redundant pick-up loops.
- **Separation of Loading Logic:** Loading actions were bifurcated into load-normal-artifact and load-delicate-artifact. This deviation cleanly enforces the rule that delicate items must have an anti-vibration pod without complicating the preconditions for standard items.

5 Comprehensive Experimental Results

The following data reflects the results obtained when solving the formulated problems across various planners and configurations.

5.1 Classical Planning (Problems 1 and 2)

Testing was performed using Fast Downward via Planutils.

Prob.	**Config.**	**Time (s)**	**Length**	**States Exp.**	**Observation**
P1	LAMA-First	0.26s	18	29	Quick satisficing plan.
P1	A* (Blind)	0.27s	18	2,007	Optimal path found.
P1	seq-opt-lmcut	N/A	Failed	N/A	Exit code 34; neg. preconditions.
P2	LAMA-First	0.28s	25	57	Suboptimal multi-agent path.
P2	A* (Blind)	0.43s	20	111,651	Optimal state-space explosion.

5.2 Hierarchical Task Networks (Problem 3)

Testing was performed using the PANDA framework.

Algorithm	**Heuristic**	**Length**	**Nodes**	**Search Time**	**Status**
PANDA (Greedy)	hhRC2(ff)	67*	76	0.001s	Solved.

**Note: Plan length includes internal task decomposition steps.*

5.3 Temporal Planning (Problem 4)

Testing was performed using Optic and Temporal Fast Downward (TFD).

Configuration	Total Time	Makespan	Evaluated	Observation
Optic (Greedy)	0.10s	85.008	66	Optimized concurrency.
Optic (Weighted A*)	0.10s	85.008	66	Identical to Greedy search.
Optic (BFS)	12.47s	189.019	38,185	Found much longer plan.
TFD (Anytime)	30s+	N/A	77,819+	Search interrupted.
TFD (Basic)	100s+	N/A	121,120+	Failed to find a plan.

6 Critical Discussion and Scalability Analysis

- **Search Space Complexity**: P2 A* (Blind) expanded 111,651 states, whereas P1 A* expanded only 2,007. This illustrates the exponential growth caused by adding multi-agent coordination and capacity constraints.
- **Heuristic Impact in Optic**: Greedy/Weighted A* found an 85s plan in 0.1s using just 66 evaluations. Conversely, BFS (Without Hill Climbing) evaluated 38,185 states to find a suboptimal 189s plan, proving that specialized temporal heuristics are vital for efficient makespans.
- **HTN Efficiency**: Problem 3 required only 76 heuristic nodes to find a complex 67-step decomposition, proving that hierarchical domain knowledge scales significantly better than declarative search.
- **Temporal Failure**: TFD evaluated over 120,000 states in 100s without finding a plan. This highlights that TFD struggles with concurrent state spaces unless the problem structure is specifically tuned for its causal graph heuristics.

7 PlanSys2 Implementation (Problem 5)

The temporal model was deployed in PlanSys2 using the `plansys2-assignment` package. Actions were implemented as C++ nodes simulating durative execution via progress variables and feedback triggers. The POPF solver generated a concurrent plan with a makespan of 83.008 seconds, and execution was verified with a "Plan Succeeded" status in the terminal logs.

8 Archive Organization

The project archive is structured as follows:

- **/Problem1**: Baseline PDDL domain and problem.
- **/Problem2**: Extended PDDL for drones and capacity system.
- **/Problem3**: HDDL files and HTN solver outputs.
- **/Problem4**: Temporal PDDL and comprehensive solver logs.
- **/Problem5**: The complete ROS 2 infrastructure for deployment.